DS HW 2

Q1. **Define an integer array. Declare an integer pointer and initialize it to point to the first element of the array. Use pointer arithmetic to traverse the array and print the elements. Modify the elements using pointer arithmetic.**

```
#include <stdio.h>

int main() {

 int a[5] = {10, 20, 30, 40, 50};

 int *p = a;

 int i;

printf("Array elements:\n");
for(i = 0; i < 5; i++)
{
   printf("%d ", *(p + i));
}

for(i = 0; i < 5; i++)
{
   *(p + i) = *(p + i) + 5;
}

printf("\nModified array:\n");
for(i = 0; i < 5; i++)
{
   printf("%d ", *(p + i));
}

return 0;


}
```

OUTPUT:

Array elements:

10 20 30 40 50

Modified array:

15 25 35 45 55

=== Code Execution Successful ===

Q2. **Calculate string length using pointers:**

```
#include <stdio.h>

int main()
{
    char str[100];
    char *p;
    int len = 0;

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    p = str;

    while(*p != '\0')
    {
        if(*p != '\n')
            len++;

        p++;
```

```
    }

    printf("Length of string = %d", len);

    return 0;
}
```

OUTPUT:

Enter a string: Hello world

Length of string = 11

=== Code Execution Successful ===

Q3. **Write a function that calculates both the sum and product of two numbers. Return these results using pointers passed as arguments.**

```
#include <stdio.h>

void calculate(int a, int b, int *sum, int *product)

 {

        *sum = a + b; *product = a * b;

}

int main() {

        int n1, n2, sum, product;

        printf("Enter two numbers: ");
        scanf("%d%d", &n1, &n2);

        calculate(n1, n2, &sum, &product);

        printf("Sum = %d\n", sum);
        printf("Product = %d\n", product);
```

```
    return 0;

}
```

OUTPUT:

Enter two numbers: 10 20

Sum = 30

Product = 200

=== Code Execution Successful ===

Q4. **Create an array of structures. Use dynamic memory allocation and a structure pointer with pointer arithmetic to access and modify members:**

```
#include <stdio.h>

 #include <stdlib.h>

struct Student {

    int id;

    char name[20];

    float marks;

};

int main() {

     int n, i;

    printf("Enter number of students: ");
    scanf("%d", &n);
    struct Student *s;

    s = (struct Student *)malloc(n * sizeof(struct Student));
```

```
    for(i = 0; i < n; i++)
    {
        printf("Enter ID Name Marks: ");
        scanf("%d%s%f", &(s+i)->id, (s+i)->name, &(s+i)->marks);
    }

    printf("\nStudent Details\n");

    for(i = 0; i < n; i++)
    {
      printf("%d %s %.2f\n",(s+i)->id, (s+i)->name, (s+i)->marks);
  }

    free(s);

    return 0;
}
```

OUTPUT:

```
Enter number of students: 2
Enter ID Name Marks: 101 Ram 85
Enter ID Name Marks: 102 Ravi 89

Student Details
101 Ram 85.00
102 Ravi 89.00
=== Code Execution Successful ===
```

Q5. Define two functions with the same signature. Declare a function pointer and use it to call both functions.

```
#include <stdio.h>
```

```
void add(int a, int b)
{
    printf("Addition = %d\n", a + b);
}

void multiply(int a, int b)
{
    printf("Multiplication = %d\n", a * b);
}

int main()
{
    void (*fp)(int, int);

    fp = add;
    fp(10, 20);

    fp = multiply;
    fp(10, 20);
    return 0;
}
```

OUTPUT:

Addition = 30

Multiplication = 200

=== Code Execution Successful ===

Q6. **Declare an integer, a pointer to it, and a double pointer. Print values and addresses at each level:**

```
#include <stdio.h>

int main() {

        int x = 25;

         int *p = &x;

        int **q = &p;

        printf("Value of x = %d\n", x);
        printf("Address of x = %p\n", (void*)&x);

        printf("Value through p = %d\n", *p);
        printf("Address stored in p = %p\n", (void*)p);

        printf("Value through q = %d\n", **q);
        printf("Address stored in q = %p\n", (void*)q);

        return 0;


}
```

OUTPUT:

Value of x = 25

Address of x = 0x7ffd6c0522d4

Value through p = 25

Address stored in p = 0x7ffd6c0522d4

Value through q = 25

Address stored in q = 0x7ffd6c0522c8

=== Code Execution Successful ===

Q7. **Sort an array of floating-point numbers using Selection Sort:**

```
#include <stdio.h>


int main()
{
    int n, i, j, min;
    float a[100], temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter the elements:\n");
    for(i = 0; i < n; i++)
        scanf("%f", &a[i]);

    for(i = 0; i < n - 1; i++)
    {
        min = i;
```

```
        for(j = i + 1; j < n; j++)
        {
            if(a[j] < a[min])
                min = j;
        }

        temp = a[i];
        a[i] = a[min];
        a[min] = temp;
    }

    printf("Sorted array:\n");
    for(i = 0; i < n; i++)
        printf("%.2f ", a[i]);

    return 0;
}
```

OUTPUT:

Enter number of elements: 5

Enter the elements:

5.6 2.4 9.1 1.2 3.8

Sorted array:

1.20 2.40 3.80 5.60 9.10

=== Code Execution Successful ===

Q8. **Define a structure Student with id, name and grade. Create an array of students and sort them based on grade:**

```
#include <stdio.h>

 struct Student

{

    int id;

    char name[20];

    float grade;

};


int main()

{

    struct Student s[10], temp;

    int n, i, j;


    printf("Enter number of students: ");

    scanf("%d", &n);


    for(i = 0; i < n; i++)

    {
```

```
        printf("Enter ID Name Grade: ");
        scanf("%d%s%f", &s[i].id, s[i].name, &s[i].grade);
    }

    for(i = 0; i < n - 1; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(s[i].grade > s[j].grade)
            {
                temp = s[i];
                s[i] = s[j];
                s[j] = temp;
            }
        }
    }

    printf("\nSorted Student Details:\n");

    for(i = 0; i < n; i++)
        printf("%d %s %.2f\n", s[i].id, s[i].name, s[i].grade);
    return 0;
}
```

OUTPUT:

Enter number of students: 3

Enter ID Name Grade: 101 Ram 8.1

Enter ID Name Grade: 102 Ravi 9.1

Enter ID Name Grade: 103 Hari 7.8

Sorted Student Details:

103 Hari 7.80

101 Ram 8.10

102 Ravi 9.10

=== Code Execution Successful ===

Q9. **Sort a set of negative numbers using Radix Sort:**

```
#include <stdio.h>
#include <stdlib.h>
void radixSort(int a[], int n)
{
    int i, exp, max = abs(a[0]);
    int output[100];

    for(i = 1; i < n; i++)
        if(abs(a[i]) > max)
            max = abs(a[i]);
```

```
for(exp = 1; max / exp > 0; exp *= 10)
{
    int count[10] = {0};

    for(i = 0; i < n; i++)
        count[(abs(a[i]) / exp) % 10]++;

    for(i = 1; i < 10; i++)
        count[i] += count[i - 1];

    for(i = n - 1; i >= 0; i--)
    {
        output[count[(abs(a[i]) / exp) % 10] - 1] = a[i];
        count[(abs(a[i]) / exp) % 10]--;
    }

    for(i = 0; i < n; i++)
        a[i] = output[i];
}

for(i = 0; i < n / 2; i++)
{
```

```
        int t = a[i];
        a[i] = a[n - 1 - i];
        a[n - 1 - i] = t;
    }
}

int main()
{
    int a[100], n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter negative numbers:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    radixSort(a, n);

    printf("Sorted array:\n");
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);
```

```
    return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
Enter negative numbers:
-45 -12 -98 -5 -63
Sorted array:
-98 -63 -45 -12 -5
=== Code Execution Successful ===
```

Q10. Write a C program to copy one array to another using pointers:

```
#include <stdio.h>
int main() {
        int a[100],b[100], n, i;
         int *p = a;
        int *q = b;
        printf("Enter number of elements: ");
        scanf("%d", &n);

        printf("Enter array elements:\n");
        for(i = 0; i < n; i++)
                scanf("%d", p + i);

        for(i = 0; i < n; i++)
                *(q + i) = *(p + i);
```

```
        printf("Copied array:\n");
        for(i = 0; i < n; i++)
                printf("%d ", *(q + i));

        return 0;
}
```

OUTPUT:

```
Enter number of elements: 5

Enter array elements:

10 20 30 40 50

Copied array:

10 20 30 40 50

=== Code Execution Successful ===
```